

O que é um Ponteiro?

O que é uma variável?

- Uma variável em um programa é algo com um nome, cujo valor pode variar.
- O compilador e o linker atribuem um bloco específico de memória dentro do computador para armazenar o valor daquela variável
- O tamanho desse bloco depende do intervalo no qual a variável pode variar
 - Por exemplo, em PCs, o tamanho de uma variável inteira é de 4 bytes e o de um inteiro longo é de 8 bytes
 - Em C, o tamanho de um tipo de variável, como um inteiro, não precisa ser o mesmo em todos os tipos de máquinas.

O que é uma variável em C?

Quando declaramos uma variável, informamos ao compilador duas coisas, o nome da variável e o tipo da variável.

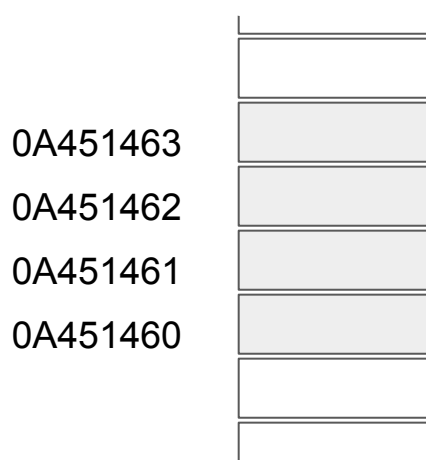
```
int k;
```

declara uma variável do tipo inteiro com o nome k

O que é uma variável em C?

```
int k;
```

Ao ver a parte “**int**” desta instrução, o compilador reserva **4 bytes** de memória (em um PC de 32 bits) para armazenar o valor do inteiro.



O que é uma variável em C?

```
int k;
```

Também configura uma tabela de símbolos. Nessa tabela, ele adiciona o símbolo `k` e o endereço relativo na memória onde esses 4 bytes foram reservados.

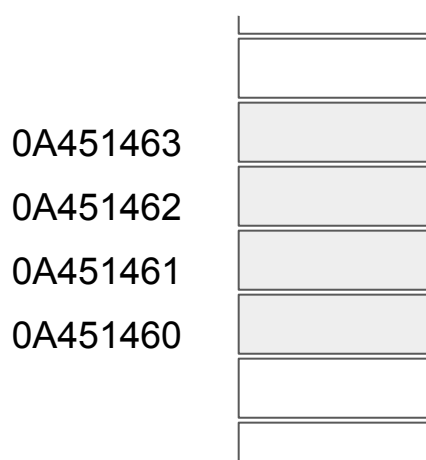


Tabela de Símbolos

k	0A451460
---	----------

O que é uma variável em C?

se mais tarde escrevermos:

```
k = 2;
```

esperamos que, em tempo de execução quando esta instrução for executada, o **valor 2** seja colocado naquele local de memória reservado para o armazenamento do valor de **k**.

Em C, nos referimos a uma variável como o inteiro k como um “objeto”.

Tabela de Símbolos

k	0A451460
---	----------

0A451463	00
0A451462	00
0A451461	00
0A451460	02

O que é uma variável em C?

Existem dois ‘valores’ associados ao objeto k.

Um é o valor do número inteiro armazenado (2)

O outro é o “valor” da localização da memória, ou seja, o endereço de k (0A451460).

Alguns textos referem-se a esses dois valores com a nomenclatura **rvalue** (valor à direita, pronunciado “ar value”) e **lvalue** (valor à esquerda, pronunciado “el value”), respectivamente.

Tabela de Símbolos

k	0A451460
---	----------

0A451463	00
0A451462	00
0A451461	00
0A451460	02

O que é uma variável em C?

Em alguns linguagens, o **lvalue** é o valor permitido no lado esquerdo do operador de atribuição '=' (ou seja, o endereço onde termina o resultado da avaliação do lado direito)

O **rvalue** é aquele que está no lado direito da instrução de atribuição, o 2.

Assim: **2 = k;** é ilegal.

Tabela de Símbolos

k	0A451460
---	----------

0A451463	00
0A451462	00
0A451461	00
0A451460	02

O que é uma variável em C?

```
int j, k;
```

```
k = 2;
```

```
j = 7;
```

```
k = j;
```

o compilador interpreta o **j** como o endereço da variável **j** (seu **lvalue**) e cria o código para copiar o valor **7** para esse endereço.

aqui o **j** é interpretado como seu **rvalue** (uma vez que está no lado direito do operador de atribuição '='). Ou seja, aqui **j** se refere ao valor armazenado no local da memória reservado para **j**, neste caso **7**.

Assim, o **7** é copiado para o endereço designado pelo **lvalor** de **k**.

Em todos esses exemplos, estamos usando inteiros de 4 bytes, portanto, todas as cópias de rvalues de um local de armazenamento para o outro são feitas copiando 4 bytes. Se estivéssemos usando números inteiros longos, estaríamos copiando 8 bytes.

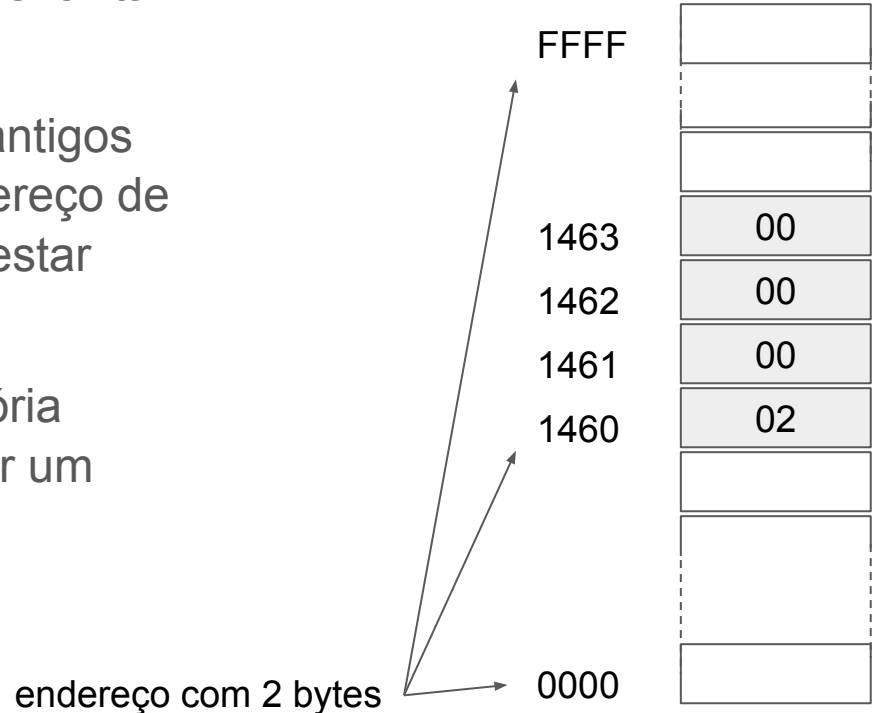
E se eu quiser uma variável projetada para conter um lvalue (um endereço)?

Variável que contém um lvalue (um endereço)

O tamanho necessário para armazenar tal valor depende do sistema.

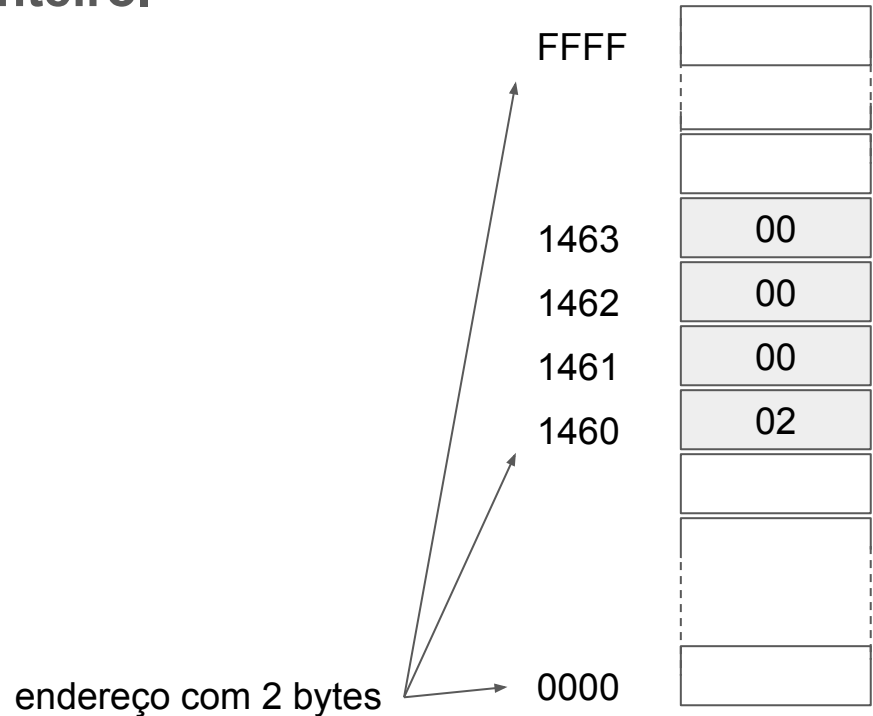
Em computadores de mesa mais antigos com 64 K de memória total, o endereço de qualquer ponto da memória pode estar contido em 2 bytes.

Os computadores com mais memória exigem mais bytes para armazenar um endereço.



Variável que contém um lvalue (um endereço)

Essa variável é chamada de **ponteiro**.

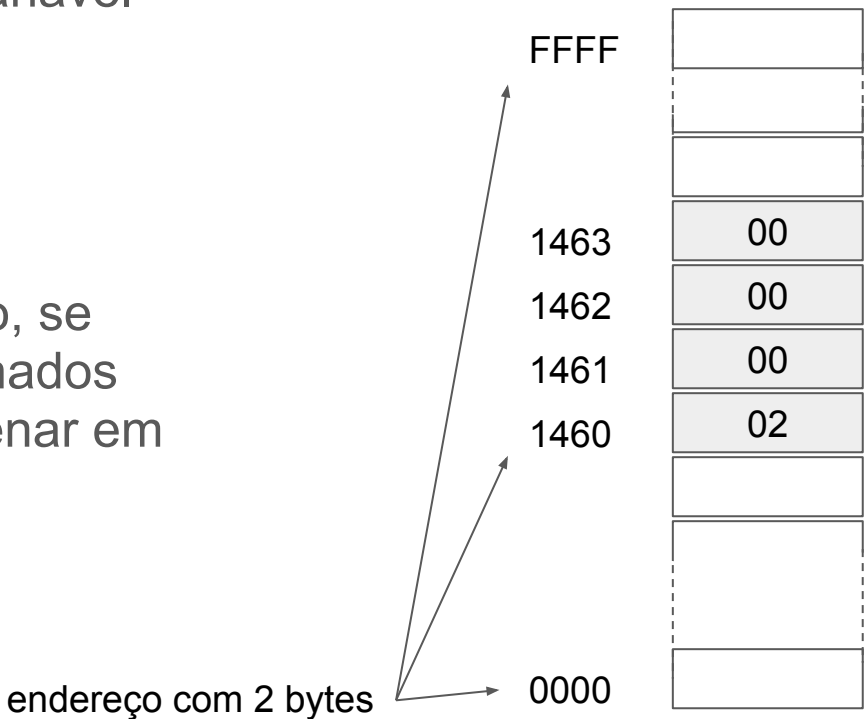


Ponteiro

Em C, quando definimos uma variável como ponteiro, fazemos isso precedendo seu nome com um **asterisco**.

Em C, também damos ao nosso ponteiro um tipo que, neste caso, se refere ao tipo de dados armazenados no endereço que iremos armazenar em nosso ponteiro.

```
int *ptr;
```



Ponteiro

```
int *ptr;
```

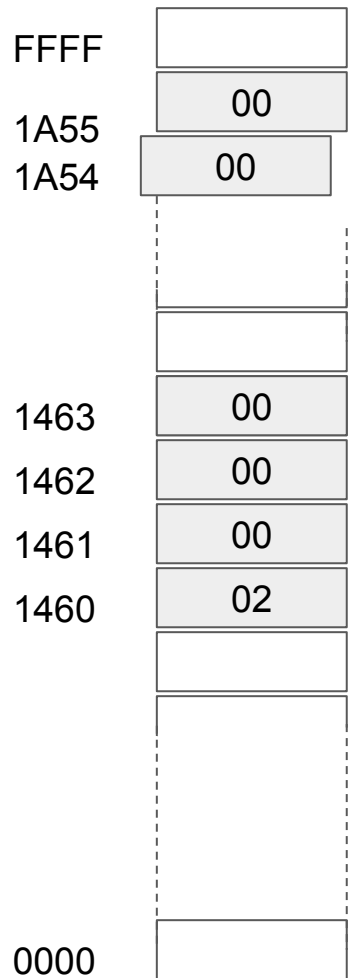
O ‘*’ informa ao compilador que queremos uma variável de ponteiro, ou seja, separar quantos bytes forem necessários para armazenar um endereço na memória.

O **int** diz que pretendemos usar nossa variável de ponteiro para armazenar o endereço de um inteiro.

Diz-se que esse ponteiro “aponta para” um número inteiro.

Tabela de Símbolos

k	1460
ptr	1A54



Ponteiro nulo

quando escrevemos

```
int k;
```

não atribuímos um valor a k.

Da mesma forma, **ptr** não tem valor, ou seja, não armazenamos um endereço nele na declaração acima.

Ele é inicializado com um valor garantido de forma que não aponte para nenhum objeto ou função C, ou seja, um ponteiro “**nulo**”.

Tabela de Símbolos

k	1460
ptr	1A54

FFFF

1A54

00

1463

00

1462

00

1461

00

1460

02

0000

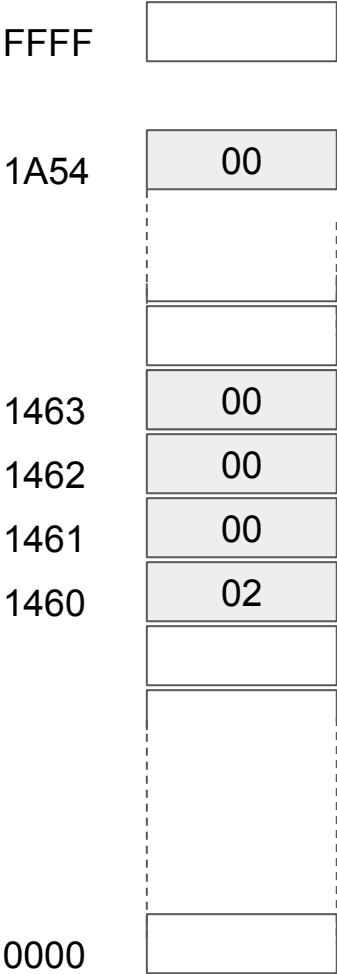
Ponteiro nulo em C

Tabela de Símbolos

k	1460
ptr	1A54

O padrão de bits real usado para um ponteiro nulo pode ou não ser avaliado como zero, pois depende do sistema específico no qual o código é desenvolvido.

Para tornar o código-fonte compatível entre vários compiladores em vários sistemas, uma macro é usada para representar um ponteiro nulo. Essa macro é chamada de **NULL**.



Ponteiro nulo em C

Atribuição

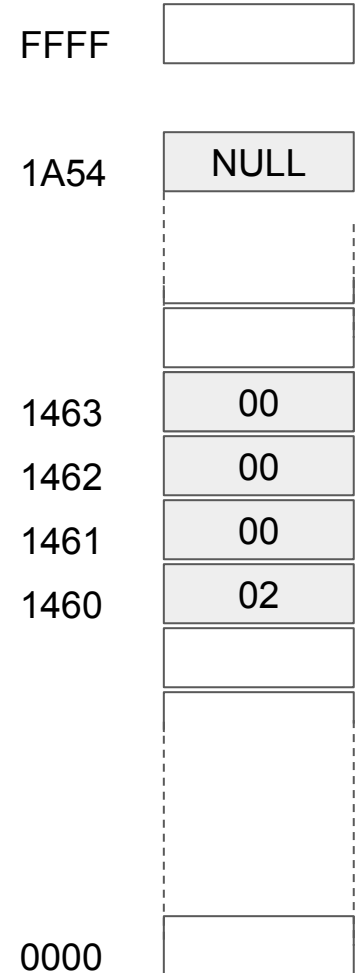
```
ptr = NULL;
```

Teste

```
if (ptr == NULL)
```

Tabela de Símbolos

k	1460
ptr	1A54



Armazenar endereço em uma variável ponteiro

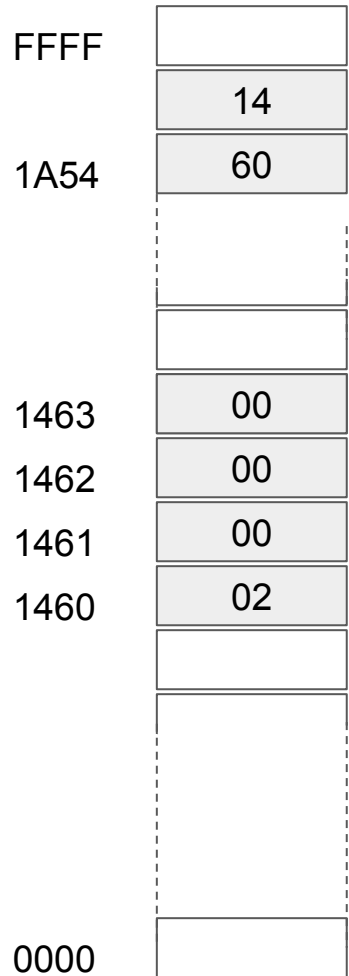
```
ptr = &k;
```

O operador & recupera o lvalue (endereço) de k, embora k esteja no lado direito do operador de atribuição '=', e o copia para o conteúdo do nosso ponteiro ptr.

Diz-se que ptr “aponta para” k.

Tabela de Símbolos

k	1460
ptr	1A54



Operador de desreferenciação *

```
*ptr = 7;
```

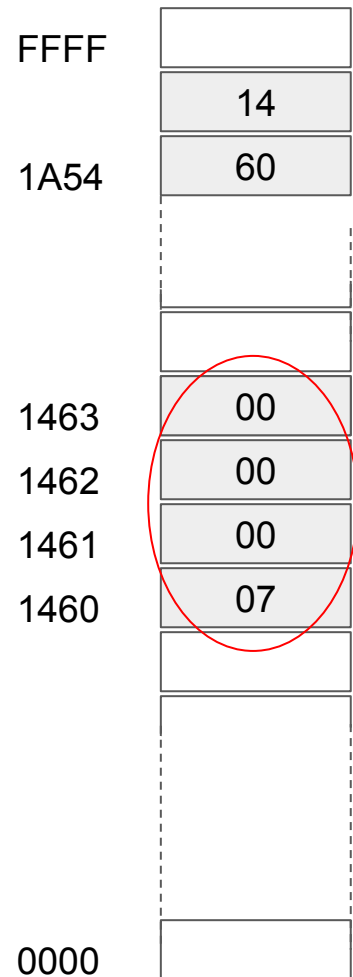
irá definir o valor de k para 7

```
printf("%d\n", *ptr);
```

imprime na tela o valor inteiro armazenado no endereço apontado por ptr;.

Tabela de Símbolos

k	1460
ptr	1A54



Exemplo de variáveis e ponteiros

```
#include <stdio.h>
```

```
int j, k;  
int* ptr;
```

```
int main(void) {  
    j = 1;  
    k = 2;  
    ptr = &k;  
    printf("j tem valor %d e está armazenado em %p\n", j, (void*)&j);  
    printf("k tem valor %d e está armazenado em %p\n", k, (void*)&k);  
    printf("ptr tem valor %p e está armazenado em %p\n",  
           (void*)ptr, (void*)&ptr);  
    printf("O valor do inteiro apontado por ptr é %d\n", *ptr);  
    return 0;  
}
```

```
gcc -Wall -std=c11 -pedantic -o prog1-1 prog1-1.c
```

Possível saída

\$./prog1-1

j tem valor 1 e está armazenado em 0x55f7cdf57018

k tem valor 2 e está armazenado em 0x55f7cdf5701c

ptr tem valor 0x55f7cdf5701c e está armazenado em 0x55f7cdf57020

O valor do inteiro apontado por ptr é 2

Esquema possível de memória

Endereço	Memória	Variável
	⋮	
0x562dcb413018	1	j
0x562dcb413020	0x562dcb413028	ptr
0x562dcb413028	2	k
	⋮	

Resumo

Uma variável é declarada dando-lhe um tipo e um nome (por exemplo, `int k;`)

Uma variável de ponteiro é declarada dando a ela um tipo e um nome (por exemplo, `int * ptr`) onde o asterisco diz ao compilador que a variável chamada `ptr` é uma variável de ponteiro e o tipo diz ao compilador para qual tipo o ponteiro deve apontar (inteiro nesse caso).

Uma vez que uma variável é declarada, podemos obter seu endereço precedendo seu nome com o operador unário `&`, como em `&k`.

Podemos “desreferenciar” um ponteiro, ou seja, referir-se ao valor para o qual ele aponta, usando o operador unário `*` como em `*ptr`.

Um “lvalue” de uma variável é o valor do seu endereço, ou seja, onde está armazenado na memória. O “rvalue” de uma variável é o valor armazenado nesta variável (naquele endereço).

Fim